SR UNIVERSITY

School of Computer Science and Artificial Intelligence

LabAssignment#19

Name of Student: Naveen.M
Enrollment No.: 2303A51436
Batch No. :21

Task1–PythontoJavaConversion
Task:ConvertaPythonprogramthatperformsfilehandlingoperationsintoan equivalent Java program using AI assistance.
Instructions:
- **PromptAItotranslateaPythonscriptthatreadsfromandwritestoafileinto Java.**
- **Ensureproperhandlingoffilestreams,exceptions,andinput/outputformatting.**
- **VerifythattheJavaprogramproducesthesamefilecontentandconsoleoutputasthe Python version.**
- **Testbothimplementationswithsampleinputfiles.**

Expected Output:
AcorrectlytranslatedandexecutableJavaprogramthatperformsthe same file handling operations as the original Python script.

Prompt:-
"create a python program that can perform file handling operations like read, write, append and delete a file. into a equivalent function in java program and ensure proper handling of streams,exception,andinput/outputformatting.makesurethatthejavaprogramshouldgive the same file content and console output as the python version and test both implementation with example inputs file.txt"

Code:-PythonImplementation

```python
# Task-1
"create a python program that can perform file handling operations like
# Python Implementation
def write_file(filename, content):
    with open(filename, 'w') as file:
        file.write(content)
        print(f"Content written to {filename}")

def read_file(filename):
    try:
        with open(filename, 'r') as file:
            content = file.read()
            print(f"Content of {filename}:\n{content}")
    except FileNotFoundError:
        print(f"{filename} not found.")
def append_file(filename, content):
    with open(filename, 'a') as file:
        file.write(content)
        print(f"Content appended to {filename}")
def delete_file(filename):
    import os
    try:
        os.remove(filename)
        print(f"{filename} has been deleted.")
    except FileNotFoundError:
        print(f"{filename} not found.")
# Example usage
filename = "file.txt"
write_file(filename, "Hello, this is a test file.\n")
read_file(filename)
append_file(filename, "This line is appended to the file.\n")
read_file(filename)
delete_file(filename)
read_file(filename)  # This will show that the file has been deleted.
```

Output:-
PythonImplementation

```
PS C:\AIAC LAB\Assignments Codes> & C:\U
thon.exe "c:/AIAC LAB/Assignments Codes/
Content written to file.txt
Content of file.txt:
Hello, this is a test file.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
Content written to file.txt
Content of file.txt:
Hello, this is a test file.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
Content of file.txt:
Hello, this is a test file.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
file.txt not found.
PS C:\AIAC LAB\Assignments Codes>
```

SR UNIVERSITY

Code:-
Java Implementation

```java
// task -01 of 19 assignmnet
// Java Implementation
import java.io.*;
class FileHandling {
    public static void writeFile(String filename, String content) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
            writer.write(content);
            System.out.println("Content written to " + filename);
        } catch (IOException e) {
            System.err.println("Error writing file: " + e.getMessage());
        }
    }
    public static void readFile(String filename) {
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            String line;
            System.out.println("Content of " + filename + ":");
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (FileNotFoundException e) {
            System.out.println(filename + " not found.");
        } catch (IOException e) {
            System.err.println("Error reading file: " + e.getMessage());
        }
    }
    public static void appendFile(String filename, String content) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename, append: true))) {
            writer.write(content);
            System.out.println("Content appended to " + filename);
        } catch (IOException e) {
            System.err.println("Error appending to file: " + e.getMessage());
        }
    }
    public static void deleteFile(String filename) {
        File file = new File(filename);
        if (file.delete()) {
            System.out.println(filename + " has been deleted.");
        } else {
            System.out.println(filename + " not found.");
        }
    }
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        String filename = "file.txt";
        writeFile(filename, content: "Hello, this is a test file.\n");
        readFile(filename);
        appendFile(filename, content: "This line is appended to the file.\n");
        readFile(filename);
        deleteFile(filename);
        readFile(filename);  // This will show that the file has been deleted.
    }
}
```

Output:-

Java Implementation

```
Content written to file.txt
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
ming\Code - Insiders\User\workspaceSto
Content written to file.txt
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
dhat.java\jdt_ws\jdt.ls-java-project\b
Content written to file.txt
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
file.txt not found.
PS C:\AIAC LAB\Assignments Codes>
```

Justification:

The provided Python and Java implementations perform the same file handling operations: writing to a file, reading from a file, appending to a file, and deleting a file. Both implementationshandleexceptionsappropriately,ensuringthattheuserisinformedifthefile is not found or if an I/O error occurs. The console output for both implementations is consistent, providing clear feedback on the operations performed. The example usage demonstrates the functionality of each operation, ensuring that both implementations yield the same results when tested with the same inputs.

Task2–C++toPythonFunction Conversion
Task:TranslateaC++programthatimplementsasortingalgorithmintoPython using AIassistance.
Instructions:
• ProvideAIwithaC++implementationofasortingtechnique(e.g.,BubbleSortor Selection Sort).
• AskAItogenerateanequivalentPython implementation.
• Ensurelogicalcorrectnessandvalidateresultsusingmultipletestcases.
• ConfirmthatthePythonversionmaintainsthesamesortingbehavior.
Expected Output:
AcorrectandexecutablePythonprogramequivalenttothegivenC++sorting implementation.

Prompt:-
"create a C++ program that implements a sorting algorithm into a equivalent function in pythonprogramandensureproperhandlingofinput/outputformatting.makesurethatthe python program should give the same sorted output as the C++ version and test both implementation with example inputs using bubble sort."

Code:-C++Implementation

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

int main() {
    vector<int> arr = {64, 34, 25, 12, 22, 11, 90};
    cout << "Original array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;
    bubbleSort(arr);
    cout << "Sorted array: ";
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;
    return 0;
}
```

Output:-C++ Implementation

```
Original array: 64 34 25 12 22 11 90
Sorted array: 11 12 22 25 34 64 90
```

Code:-PythonImplementation

```python
# Python Implementation
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
print("\nOriginal array:", arr)
bubble_sort(arr)
print("Sorted array:", arr)
```

Output:-PythonImplementation

```
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
```

Justification:
The Python implementation of bubble sort follows the same logic as the C++ version, ensuring the same sorted output. Theinput and output formatting is consistent between both implementations,allowingforeasytestingwiththesameexampleinputs.andtheoutputwill be the same for both implementations, demonstrating that the sorting algorithm works correctly in both languages.

Task3–SQLtoPandas Query
Task:ConvertaSQLqueryinvolvingGROUPBYandaggregatefunctionsintoan equivalent Pandas Data Frame operation.
Instructions:
• ProvideAIwithaSQLquerythatcalculatesaggregatevalues(e.g.,totalsalesper department).
• PromptAItogenerateequivalentPandascodeusinggroupby()andaggregation functions.
• Testthegenerated codeusingasampledataset.
• VerifythatbothSQLandPandasoutputsmatch.
Expected Output:
AnaccuratePandasDataFrameoperationthatproducesresultsequivalenttothegiven SQL query with aggregation.

Prompt:-
"create a SQL query that involves group by and aggregate functions into an equivalent functioninpandasdataframeandensureproperhandlingofdatamanipulationandoutput formatting.makesurethatthepandasimplementationshouldgivethesameresultsasthe SQL query and test both implementations with example inputs using a sample dataset."

Code:-SQL Implementation

```sql
-- SQLite

SELECT department, COUNT(*) AS employee_count, AVG(salary) AS average_salary
FROM employees
GROUP BY department;
```

Output:-SQL Implementation

```
  department salary
                count     mean
0    Finance     2  62500.0
1         HR     2  52500.0
2         IT     2  85000.0
```

Code:-PythonImplementation

```python
# Pandas Implementation
import pandas as pd
# Sample dataset
data = {
    'department': ['HR', 'IT', 'Finance', 'IT', 'HR', 'Finance'],
    'salary': [50000, 80000, 60000, 90000, 55000, 65000]
}
df = pd.DataFrame(data)
# Group by department and calculate aggregate functions
result = df.groupby('department').agg({
    'salary': ['count', 'mean']
}).reset_index()
print(result)
```

Output:-PythonImplementation

```
  department salary
                count     mean
0    Finance     2  62500.0
1         HR     2  52500.0
2         IT     2  85000.0
```

Justification:

The provided code snippets demonstrate the implementation of file handling operations in Python and Java, a bubble sort algorithm in C++ and Python, and a SQL query for grouping and aggregating data alongside its equivalent implementation using pandas in Python. Each implementationisdesignedtoproducethesameoutput,ensuringconsistencyacrossdifferent programming languages and tools. The examples are tested with sample inputs to validate their correctness and functionality.

Task4–Real-TimeApplication:Multi-LanguageSnippetConverter Scenario: Studentswillselectapracticalreal-worldapplicationrequiringthesamelogicintwo different programming languages.
Example:
• ConvertingaRESTAPIimplementation fromPython(Flask)toJava(SpringBoot).
• TranslatingadataprocessingscriptfromPythontoC++forperformance optimization.
Instructions:
• ProvideAIwithsourcecodeinoneprogramminglanguage.
• PromptAItoconvert thecodeintoanotherchosenlanguage.
• Executeandtestbothversionstoensureidenticalfunctionality.
• Documentdifferencesobservedinsyntax,libraries,andexecutionbehavior.
Expected Output:
Functionallyequivalentandexecutablecodesnippetsimplementedintwodifferent programming languages, along with documented observations.

Prompt:-
"createaPythonscriptthatcalculatesthestandard deviationofalargelistofnumbers.While Python is great for prototyping, I need to convert this into C++ to optimize for execution speed and memory efficiency. Please provide a functionally equivalent C++ version. Ensure the logic for the mean and variance remains identical, and document the key differences in syntax and memory management between the two."

Code:-PythonImplementation

```
# Task-4
# "create a Python script that calculates the standard deviat
# Python Implementation
import math
from numpy import std
from numpy import std
from py_compile import main
def calculate_standard_deviation(numbers):
    n = len(numbers)
    if n == 0:
        return 0
    mean = sum(numbers) / n
    variance = sum((x - mean) ** 2 for x in numbers) / n
    return math.sqrt(variance)
# Example usage
numbers = [10, 12, 23, 23, 16, 23, 21, 16]
std_dev = calculate_standard_deviation(numbers)
print(f"\n Standard Deviation: {std_dev}")
```

Output:-PythonImplementation

```
Standard Deviation: 4.898979485566356
```

Code:-

```
# C++ Implementation
#include <iostream>
#include <vector>
#include <cmath>
double calculateStandardDeviation(const std::vector<double>& numbers) {
    int n = numbers.size();
    if (n == 0) {
        return 0;
    }
    double mean = 0;
    for (double num : numbers) {
        mean += num;
    }
    mean /= n;
    double variance = 0;
    for (double num : numbers) {
        variance += (num - mean) * (num - mean);
    }
    variance /= n;
    return std::sqrt(variance);
}
int main() {
    std::vector<double> numbers = {10, 12, 23, 23, 16, 23, 21, 16};
    double stdDev = calculateStandardDeviation(numbers);
    std::cout << "Standard Deviation: " << stdDev << std::endl;
    return 0;
}
```

Output:-

```
Standard Deviation: 4.89898
```

Justification:-

The above code calculates the standard deviation of a set of numbers using C++. It first computes the mean of the numbers, then calculates the variance by summing the squared differences from the mean, and finally takes the square root of the variance to get the standarddeviation.Thecodeisefficientandstraightforward,makingiteasytounderstand andmaintain.anditdemonstratestheuseofbasic C++featuressuchasvectors,loops,and functions.

Task-5:(BasicFunctionTranslation(PythontoC++)
TranslateasimplePythonfunctionthatchecks whetheranumberisevenoroddinto C++ using AI assistance.
Instructions:
- **ProvideAIwithabasicPythonfunctionthatdeterminesifanumberisevenorodd.**
- **PromptAItogenerateequivalentC++code.**
- **Ensurecorrectsyntax,inputhandling,andoutputformatting.**
- **CompileandruntheC++programtoverifyitproducesthesameresultasthePython version.**
- **Testwithatleastthreedifferentinputvalues.**

Expected Output:
AsimpleandexecutableC++programequivalenttothegivenPythonfunction, correctly identifying whether a number is even or odd for all test cases.

Prompt:-
"createatranslationofasimplePythonfunctionthatchecks whetheragivennumberiseven oroddintoanequivalent functioninc++.ensurethatthelogicremainsidenticalandthatthe output formatting is consistent between both implementations. test both functions with a range of input values to confirm they produce the same results."

Code:-

```
# Task-5
# "create a translation of a simp
# Python Implementation
from ast import main
def check_even_odd(number):
    if number % 2 == 0:
        return "Even"
    else:
        return "Odd"
# Example usage
for num in range(1, 11):
    result = check_even_odd(num)
    print(f"{num} is {result}")
```

Output:-

```
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
6 is Even
7 is Odd
8 is Even
9 is Odd
10 is Even
```

Code:-

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <cmath>
std::string checkEvenOdd(int number) {
    if (number % 2 == 0) {
        return "Even";
    } else {
        return "Odd";
    }
}
int main() {
    for (int num = 1; num <= 10; num++) {
        std::string result = checkEvenOdd(num);
        std::cout << num << " is " << result << std::endl;
    }
    return 0;
}
```

Output:-

```
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
6 is Even
7 is Odd
8 is Even
9 is Odd
10 is Even
```

Justification:-

The provided code snippet demonstrates a simple Java Spring Boot REST API application. The`SpringBootApiApplication`classservesastheentrypointfortheapplication,whilethe `ApiController` class defines two endpoints: one for a welcome message and another for performingcalculations(squareandcube)basedonapathvariable.Theuseof`Map`allows for flexible responses, and the annotations like `@RestController` and `@GetMapping` facilitatethecreationofRESTful endpoints.ThisstructureistypicalforabasicSpringBoot application, making it easy to understand and extend in the future.